Investigacion: Capacidades Multimedia

Imagenes y Audio en Telegram Commander

v2.0 — Incluye evaluacion de servicios OpenAI (ChatGPT) para voz

Fecha	Tipo	Componente	Autor
2026-03-03	**Investigacion tecnica**	**telegram-commander.js**	**Claude Code (Guru)**

1. Resumen Ejecutivo

Se investigo la factibilidad de agregar al **Telegram Commander** la capacidad de recibir e interpretar **imagenes** y **mensajes de audio/voz** enviados desde Telegram, para que Claude pueda procesarlos como parte de su flujo de trabajo.

SI	SI	SI	NO
IMAGENES VIABLES	AUDIO VIABLE (TRANSCRIPCION)	RESPUESTA POR VOZ (TTS)	AUDIO NATIVO EN CLAUDE API

Conclusion principal

Las imagenes se pueden integrar llamando directamente a la API de Anthropic con content blocks multimodales. El audio requiere un paso intermedio de transcripcion antes de enviar el texto a Claude.

Novedad v2:

OpenAI ofrece modelos de nueva generacion (GPT-4o Transcribe, GPT-4o Mini TTS) que mejoran significativamente la calidad y reducen costos hasta un 50% respecto a Whisper legacy. Ademas, la respuesta por voz (TTS) es viable y nativa para Telegram.

Cambios en v2.0 respecto a v1.0

- Seccion 4.3 ampliada: 3 modelos de transcripcion OpenAI + 3 alternativas del mercado
- Nueva seccion 5: Respuesta por voz (Text-to-Speech) — evaluacion completa
- Nueva seccion 6: Evaluacion de la Realtime API de ChatGPT
- Nueva seccion 7: Comparativa detallada de costos por proveedor
- Arquitectura actualizada con flujo bidireccional (voz-in + voz-out)
- Proyeccion de costos mensuales por escenario de uso

2. Estado Actual del Commander

2.1 Punto de bloqueo identificado

El `telegram-commander.js` actualmente descarta todo mensaje que no sea texto plano:

```
// telegram-commander.js, linea 2009-2010
const text = msg.text;
if (!text) continue;  // ← AQUI se descartan fotos, audio, documentos, etc.
```

2.2 Infraestructura existente aprovechable

ARCHIVO	FUNCIONALIDAD	RELEVANCIA
`telegram-image-utils.js`	Envio de imagenes PNG a Telegram (`sendTelegramPhoto`)	Alta — helpers HTTP reutilizables
`telegram-commander.js`	`telegramPost()` — helper HTTP para API de Telegram	Alta — ya maneja requests multipart
`telegram-config.json`	Bot token + chat ID centralizados	Alta — configuracion lista
`executeClaude()`	Spawn de `claude -p` via stdin	Media — solo acepta texto

3. Imagenes desde Telegram

3.1 Factibilidad: ALTA

Veredicto: Totalmente viable

La API de Claude soporta imagenes como content blocks (base64, URL o Files API). Telegram provee las fotos como JPEG, formato nativamente compatible.

3.2 Formatos soportados por Claude API

FORMATO	MIME	LIMITE	FUENTE TELEGRAM
JPEG	`image/jpeg`	5 MB	msg.photo[]
PNG	`image/png`	5 MB	msg.document
GIF	`image/gif`	5 MB	msg.document
WebP	`image/webp`	5 MB	msg.sticker

3.3 Flujo tecnico propuesto

1. **Detectar imagen** — Verificar `msg.photo` (array de tamanios) o `msg.document` con MIME de imagen

↓

2. **Obtener file_id** — Tomar el ultimo elemento de `msg.photo[]` (mayor resolucion) o `msg.document.file_id`

↓

3. **Descargar archivo** — `GET /bot{TOKEN}/getFile?file_id=...` → obtener `file_path` → `GET /file/bot{TOKEN}/{file_path}`

↓

4. **Codificar en base64** — Convertir el buffer binario a string base64 y determinar media_type

↓

5. **Llamar API Anthropic** — `POST /v1/messages` con content blocks `[{type:"image"}, {type:"text"}]`

↓

6. **Devolver respuesta** — Enviar la respuesta de Claude al chat de Telegram

3.4 Limitacion critica: claude -p no acepta imagenes

El CLI `claude -p` solo acepta texto via stdin

Para enviar imagenes a Claude, se debe llamar directamente a la API REST de Anthropic (`https://api.anthropic.com/v1/messages`) en lugar de usar `executeClaude()`. Esto requiere tener `ANTHROPIC_API_KEY` disponible en el entorno del hook.

3.5 Ejemplo de llamada a la API

```
// Llamada directa a Anthropic API con imagen
const body = {
  model: "claude-sonnet-4-6",
  max_tokens: 2048,
  messages: [{
    role: "user",
    content: [
      {
        type: "image",
```

```
          source: {
            type: "base64",
            media_type: "image/jpeg",
            data: imageBase64  // Buffer.toString("base64")
          }
        },
        {
          type: "text",
          text: caption || "Describe esta imagen y responde en espanol."
        }
      ]
    }]
  };
```

4. Audio / Voz desde Telegram (Transcripcion) ACTUALIZADO v2

4.1 Factibilidad: ALTA (revisada al alza)

Veredicto: Altamente viable con OpenAI

La API de Claude NO soporta audio como input, pero OpenAI ofrece **tres modelos de transcripcion** con precios desde **$0.003/min** y calidad excelente en espanol. La factibilidad se eleva de MEDIA a ALTA gracias a los nuevos modelos GPT-4o Transcribe (marzo 2025).

4.2 Formato de audio en Telegram

TIPO DE MENSAJE	CAMPO	FORMATO	CODEC
Voice message (microfono)	`msg.voice`	`.ogg`	Opus
Audio file (archivo)	`msg.audio`	`.mp3`, `.m4a`, etc.	Varios
Video note (circulo)	`msg.video_note`	`.mp4`	H.264

4.3 Modelos de transcripcion OpenAI (ChatGPT) NUEVO

OpenAI ofrece actualmente tres modelos de Speech-to-Text, todos via el mismo endpoint:

MODELO	ID	PRECIO/MIN	ACEPTA .OGG	CALIDAD	RECOMENDACION
GPT-4o Transcribe	`gpt-4o-transcribe`	$0.006	Si	Mejor WER — menor tasa de error, mejor manejo de acentos	Recomendado (calidad)
GPT-4o Mini Transcribe	`gpt-4o-mini-transcribe`	**$0.003**	Si	Buena — 89% menos alucinaciones vs legacy	Recomendado (costo)
Whisper (legacy)	`whisper-1`	$0.006	Si	Baseline — modelo V2 large	Superado

Ventajas clave de los nuevos modelos (marzo 2025+)

- **gpt-4o-transcribe**: menor Word Error Rate (WER), mejor manejo de ruido y acentos hispanohablantes
- **gpt-4o-mini-transcribe**: 50% mas barato que Whisper, 89% menos alucinaciones (snapshot dic 2025)
- **Speaker diarization** disponible con variante `gpt-4o-transcribe-diarize` (mismo precio)
- Todos usan el mismo endpoint: `POST /v1/audio/transcriptions`

4.4 Alternativas del mercado NUEVO

SERVICIO	MODELO	PRECIO/MIN	ACEPTA .OGG	FREE TIER	ESPANOL	STREAMING
OpenAI	gpt-4o-mini-transcribe	**$0.003**	Si	$5 credito	Excelente	No (batch)
Deepgram	Nova-3	$0.0043	Si	$200 credito	Excelente (+20% WER)	Si
Google Cloud	STT V2 standard	$0.016	Si	60 min/mes	125+ langs	Si
Google Cloud	STT V2 enhanced	$0.024-$0.036	Si	60 min/mes	125+ langs	Si
AWS	Transcribe	$0.024	Limitado	60 min/mes (12 meses)	Si	Si
AssemblyAI	Universal-2	$0.0065	Si	Freemium	Si	Si

Veredicto comparativa

OpenAI gpt-4o-mini-transcribe

es la mejor opcion para nuestro caso: el mas barato ($0.003/min), acepta OGG nativamente, excelente calidad en espanol, y ya necesitamos `OPENAI_API_KEY` igualmente. Deepgram es alternativa competitiva si se necesita streaming en el futuro.

4.5 Flujo tecnico propuesto (actualizado)

1. **Detectar audio** — Verificar `msg.voice` o `msg.audio`

↓

2. **Descargar archivo** — `getFile` → descargar el `.ogg` como Buffer

↓

3. **Transcribir** — `POST /v1/audio/transcriptions` con modelo `gpt-4o-mini-transcribe` (o `gpt-4o-transcribe` para mayor calidad)

↓

4. **Enviar texto a Claude** — Usar `executeClaude(transcripcion)` via `claude -p` normalmente

↓

5. **Devolver respuesta** — Texto al chat + opcionalmente voz (ver seccion 5)

4.6 Ejemplo de transcripcion (actualizado)

```
// Transcripcion con OpenAI GPT-4o Mini Transcribe (Node.js puro)
async function transcribeAudio(audioBuffer, filename) {
  const boundary = "———TranscribeBoundary" + Date.now().toString(36);
  let body = "";
  body += "--" + boundary + "\r\n";
  body += 'Content-Disposition: form-data; name="model"\r\n\r\n';
  body += 'gpt-4o-mini-transcribe\r\n';  // ← NUEVO: 50% mas barato que whisper-
1
  body += "--" + boundary + "\r\n";
  body += 'Content-Disposition: form-data; name="language"\r\n\r\nes\r\n';

  const pre = Buffer.from(
    body + "--" + boundary + "\r\n" +
    'Content-Disposition: form-data; name="file"; filename="' + filename +
'"\r\n' +
    'Content-Type: audio/ogg\r\n\r\n'
  );
  const post = Buffer.from("\r\n--" + boundary + "--\r\n");
  const payload = Buffer.concat([pre, audioBuffer, post]);

  // POST a https://api.openai.com/v1/audio/transcriptions
  // Headers: Authorization: Bearer {OPENAI_API_KEY}
  // Content-Type: multipart/form-data; boundary={boundary}
```

```
  return transcriptionText;
}
```

5. Respuesta por Voz (Text-to-Speech) NUEVO en v2

5.1 Factibilidad: ALTA

Veredicto: Totalmente viable y natural para Telegram

OpenAI ofrece TTS con salida en formato Opus — el mismo codec nativo de Telegram para mensajes de voz. Esto permite que el bot **responda con audio** ademas de texto, creando una experiencia conversacional completa (voz-in → voz-out).

5.2 Modelos de Text-to-Speech disponibles

MODELO	ID	PRECIO	VOCES	CALIDAD	FEATURE CLAVE
TTS Standard	`tts-1`	$15/1M chars	6 voces	Buena	Baja latencia
TTS HD	`tts-1-hd`	$30/1M chars	6 voces	Alta	Mayor fidelidad
GPT-4o Mini TTS	`gpt-4o-mini-tts`	$12/1M output tokens	**13 voces**	**Estudio (48kHz)**	**Dirigible** — se puede controlar tono, emocion, ritmo

5.3 Voces disponibles (gpt-4o-mini-tts)

VOZ	CARACTER	RECOMENDACION PARA BOT
`nova`	Femenina, joven, energetica	Recomendada — natural en espanol
`coral`	Femenina, calida, profesional	Recomendada — tono asistente
`alloy`	Neutral, versatil	Buena opcion
`onyx`	Masculina, grave, seria	Alternativa
`sage`	Neutra, experta	Alternativa
`ash`, `ballad`, `echo`, `fable`, `shimmer`, `verse`, `marin`, `cedar`	Diversas personalidades	Disponibles

5.4 Dirigibilidad (feature unico de gpt-4o-mini-tts)

Control fino del estilo de habla

A diferencia de los modelos legacy, `gpt-4o-mini-tts` acepta instrucciones de estilo: "*Habla en tono amigable y profesional, con ritmo moderado, como un asistente tecnico.*" Esto permite adaptar la personalidad del bot sin cambiar de voz.

5.5 Flujo tecnico: respuesta por voz

1. **Claude genera respuesta** — Texto normal del flujo existente

↓

2. **Sintetizar voz** — `POST /v1/audio/speech` con `model: "gpt-4o-mini-tts"`, `response_format: "opus"`

↓

3. **Enviar como voice message** — `sendVoice` de Telegram Bot API con el buffer Opus

5.6 Ejemplo de sintesis de voz

```
// Text-to-Speech con GPT-4o Mini TTS (Node.js puro)
async function textToSpeech(text) {
```

```
  const body = JSON.stringify({
    model: "gpt-4o-mini-tts",
    input: text,
    voice: "nova",           // voz natural en espanol
    response_format: "opus",  // formato nativo de Telegram
    instructions: "Habla en espanol con tono profesional y amigable."
  });

  // POST a https://api.openai.com/v1/audio/speech
  // Headers: Authorization: Bearer {OPENAI_API_KEY}
  // Content-Type: application/json
  // Response: binary audio (Opus)
  return audioBuffer;  // enviar a Telegram via sendVoice
}
```

5.7 Consideraciones de UX para respuesta por voz

Cuanado responder con voz

Recomendacion: responder con voz **solo cuando el usuario envio un voice message**. Si envio texto, responder con texto. Esto respeta la preferencia del canal elegido por el usuario.

Alternativa: agregar un comando `/voice on` / `/voice off` para que el usuario elija.

Limites para TTS

- Maximo 2,000 tokens por request (aprox. 1,500 palabras en espanol)
- Respuestas muy largas deben truncarse o enviarse como texto + resumen en voz
- Latencia: 1-3s para respuestas cortas, aceptable para Telegram

6. Evaluacion: Realtime API de ChatGPT NUEVO en v2

6.1 Que es la Realtime API

La **Realtime API** de OpenAI permite conversaciones de voz bidireccionales en tiempo real via WebSocket. Fue lanzada en produccion como `gpt-realtime` en octubre 2025.

6.2 Precios

MODELO	AUDIO INPUT	AUDIO OUTPUT	COSTO APROX./MIN
`gpt-realtime`	$32/1M tokens	$64/1M tokens	~$0.30/min
`gpt-4o-mini-realtime-preview`	$10/1M tokens	$20/1M tokens	~$0.10/min

6.3 Evaluacion para nuestro caso de uso

NO recomendada para Telegram Commander

La Realtime API esta disenada para **conversaciones de voz continuas** con baja latencia (tipo llamada telefonica), no para mensajes discretos de Telegram.

Razones para descartarla:

- **50-100x mas cara** que el pipeline Transcribe + TTS ($0.30/min vs $0.006/min)
- Requiere **WebSocket persistente** — overhead innecesario para mensajes esporadicos
- El modelo **no es Claude** — la respuesta la genera GPT-4o, no Claude. Se pierde el contexto del proyecto
- **No se integra** con el flujo existente de `executeClaude()`
- Beneficio principal (baja latencia) **irrelevante** para Telegram (los mensajes ya son asincronos)

Arquitectura correcta: pipeline separado

Usar **Transcribe** (voz → texto) + **Claude** (texto → respuesta) + **TTS** (respuesta → voz) es la arquitectura optima porque:

- Claude procesa la logica y el contexto del proyecto
- OpenAI solo maneja la conversion de/hacia audio
- Costo 50-100x menor
- Cada componente es reemplazable independientemente

7. Comparativa de Costos por Escenario NUEVO en v2

7.1 Costo de transcripcion (voz → texto)

ESCENARIO	VOLUMEN	OPENAI MINI	OPENAI 4O	DEEPGRAM	GOOGLE STT	AWS
Uso ligero	100 msgs (50 min/mes)	$0.15	$0.30	$0.22	$0.80	$1.20
Uso moderado	1,000 msgs (500 min/mes)	$1.50	$3.00	$2.15	$8.00	$12.00
Uso intensivo	10,000 msgs (5,000 min/mes)	$15.00	$30.00	$21.50	$80.00	$120.00

* Asumiendo mensajes de voz promedio de 30 segundos. Precios en USD.

7.2 Costo de TTS (texto → voz, opcional)

ESCENARIO	RESPUESTAS	GPT-4O-MINI-TTS	TTS-1	TTS-1-HD
Uso ligero	100 respuestas cortas	~$0.50	~$0.75	~$1.50
Uso moderado	1,000 respuestas cortas	~$5.00	~$7.50	~$15.00

7.3 Costo total mensual proyectado (pipeline completo)

$0.65
USO LIGERO (100 MSGS + TTS)

$6.50
USO MODERADO (1K MSGS + TTS)

$65
USO INTENSIVO (10K MSGS + TTS)

* Pipeline: gpt-4o-mini-transcribe ($0.003/min) + gpt-4o-mini-tts ($12/1M tokens). No incluye costo de Claude API.

8. Arquitectura Propuesta (v2) ACTUALIZADO

```
Telegram                    Commander                    APIs externas
═══════                     ═══════                      ═══════════

[Foto JPEG] ──msg.photo──> detectar tipo ──base64──> Anthropic API /v1/messages
                                |                       (content: image + text)
                                |                            |
[Voice .ogg] ──msg.voice──> detectar tipo ──buffer──> OpenAI /v1/audio/transcriptions
                                |                       (gpt-4o-mini-transcribe)
                                |                            |
                                |                        texto transcrito
                                |                            |
[Texto]     ──msg.text───> detectar tipo             claude -p (stdin)
                                |                            |
                               <────── respuesta <──────────┘
                                |
                                ├── es voice? ──SI──> OpenAI /v1/audio/speech
                                |                     (gpt-4o-mini-tts, opus)
                                |                            |
[Chat TG]   <── sendVoice ──────────────────────────────────┘
[Chat TG]   <── sendMessage──┘ (texto, si no era voice)


Dependencias:
┌──────────────────────────────────────────────────────────────────┐
│  ANTHROPIC_API_KEY  (para imagenes, API directa de Claude)        │
│  OPENAI_API_KEY     (para transcripcion + TTS, pipeline de audio) │
└──────────────────────────────────────────────────────────────────┘


Pipeline de audio completo:
┌─────────┐    ┌──────────────┐    ┌─────────┐    ┌──────────┐    ┌─────────┐
│ Telegram │──>│ Transcribe   │──>│ Claude  │──>│ TTS      │──>│Telegram │
│ Voice    │    │ (OpenAI STT) │    │ (texto) │    │ (OpenAI) │    │ Voice   │
└─────────┘    └──────────────┘    └─────────┘    └──────────┘    └─────────┘

   .ogg            texto              texto           .opus          .ogg
  $0.003/min                      (costo Claude)   ~$0.01/resp
```

9. Dependencias y Requisitos

REQUISITO	PARA	ESTADO ACTUAL	ACCION
`ANTHROPIC_API_KEY`	Imagenes (API directa)	Verificar	Agregar a `telegram-config.json` o variable de entorno
`OPENAI_API_KEY`	Transcripcion + TTS	No configurada	Obtener key de OpenAI y agregar a `telegram-config.json`
Node.js `https`	Todas las llamadas HTTP	Disponible	Ya incluido en Node.js (sin deps externas)
Telegram Bot API	`getFile`, `sendVoice`, `sendMessage`	Bot activo	Ya configurado en `telegram-config.json`
`ffmpeg` (opcional)	Conversion OGG → MP3 si hay problemas	Opcional	Solo si el API no acepta bien el OGG/Opus de Telegram

10. Estimacion de Esfuerzo ACTUALIZADO

FEATURE	COMPLEJIDAD	ARCHIVOS A MODIFICAR	DEPENDENCIA EXTERNA
Imagenes → Claude	Media	`telegram-commander.js` (handler + API call)	`ANTHROPIC_API_KEY`
Audio/Voz → Transcripcion → Claude	Media	`telegram-commander.js` (handler + OpenAI transcribe + Claude)	`OPENAI_API_KEY`
Respuesta por voz (TTS) NUEVO	Media	`telegram-commander.js` (TTS call + sendVoice)	`OPENAI_API_KEY` (misma key)
Caption como prompt	Baja	`telegram-commander.js` (extraer `msg.caption`)	Ninguna

Nota sobre esfuerzo de TTS

La complejidad se redujo de Media-Alta a Media porque: (1) `OPENAI_API_KEY` ya se configura para la transcripcion, (2) el formato Opus sale directo del API sin conversion, (3) `sendVoice` de Telegram acepta un buffer binario directamente. Son ~30 lineas de codigo adicionales.

11. Riesgos y Mitigaciones

Costo por uso de APIs

Anthropic API (imagenes): ~$0.004/imagen de 1000x1000px.
OpenAI Transcribe: $0.003-$0.006/minuto (segun modelo elegido).
OpenAI TTS: ~$0.01-$0.02/respuesta corta.
Mitigacion: Limitar tamanio de imagenes/audio, rate-limiting por chat, mostrar aviso de costo. Usar `gpt-4o-mini-transcribe` (50% mas barato) por defecto.

Timeout en transcripcion/TTS

Audios largos pueden tardar mas en transcribir; respuestas largas tardan mas en sintetizar.
Mitigacion: Limitar a 2 minutos de audio, timeout de 30s, truncar TTS a 2000 tokens, enviar "Transcribiendo..." / "Generando audio..." al chat con `sendChatAction`.

Seguridad de API keys

Las keys no deben quedar en el codigo fuente.
Mitigacion: Usar `telegram-config.json` (ya en .gitignore) o variables de entorno del sistema. Una sola `OPENAI_API_KEY` sirve para transcripcion Y TTS.

Vendor lock-in con OpenAI NUEVO

Usar OpenAI para transcripcion Y TTS crea dependencia en un solo proveedor.
Mitigacion: La arquitectura de pipeline permite reemplazar cualquier componente: si OpenAI sube precios o degrada servicio, se puede migrar transcripcion a Deepgram o TTS a otro proveedor sin cambiar el flujo general. Las funciones `transcribeAudio()` y `textToSpeech()` encapsulan la dependencia.

12. Recomendaciones ACTUALIZADO

1. Implementar imagenes primero

Es la feature mas directa y de mayor valor inmediato. Permite enviar capturas de pantalla, diagramas, errores visuales y que Claude los interprete en contexto. Solo requiere `ANTHROPIC_API_KEY` que probablemente ya este disponible.

2. Audio (transcripcion) como segunda fase

Usar **`gpt-4o-mini-transcribe`** como modelo por defecto: 50% mas barato que Whisper, 89% menos alucinaciones, excelente calidad en espanol. Opcionalmente exponer un comando `/quality high` para usar `gpt-4o-transcribe` cuando se necesite maxima precision.

3. Respuesta por voz (TTS) como tercera fase NUEVO

Usar **`gpt-4o-mini-tts`** con voz nova y formato opus. Activar automaticamente cuando el usuario envie un voice message (espejo del canal). La misma `OPENAI_API_KEY` sirve, no hay dependencia adicional. Agregar instrucciones de estilo: "*Habla en espanol con tono profesional y amigable*".

4. UX: Indicadores de progreso

Enviar mensajes intermedios al chat: "Analizando imagen..." / "Transcribiendo audio..." / "Generando respuesta de voz...". Telegram tiene `sendChatAction(typing)` y `sendChatAction(record_voice)` que muestran indicadores nativos automaticamente.

5. NO usar la Realtime API de ChatGPT

La Realtime API ($0.30/min) es 50-100x mas cara y esta disenada para conversaciones continuas, no para mensajes discretos de Telegram. Ademas, reemplazaria a Claude como motor de respuesta, perdiendo todo el contexto del proyecto. El pipeline Transcribe + Claude + TTS es superior en todos los aspectos para este caso de uso.

13. Fuentes

FUENTE	TIPO
Claude API Vision Docs — `platform.claude.com/docs/en/build-with-claude/vision`	Documentacion oficial
Telegram Bot API — `core.telegram.org/bots/api`	Documentacion oficial
OpenAI Whisper API — `platform.openai.com/docs/guides/speech-to-text`	Documentacion oficial
OpenAI GPT-4o Transcribe — `platform.openai.com/docs/models/gpt-4o-transcribe`	Documentacion oficial
OpenAI GPT-4o Mini TTS — `platform.openai.com/docs/models/gpt-4o-mini-tts`	Documentacion oficial
OpenAI API Pricing — `openai.com/api/pricing`	Precios oficiales
OpenAI Realtime API — `openai.com/index/introducing-gpt-realtime`	Documentacion oficial
Deepgram Nova-3 — `deepgram.com/learn/best-speech-to-text-apis-2026`	Comparativa
Google Cloud STT Pricing — `cloud.google.com/speech-to-text/pricing`	Precios oficiales
AWS Transcribe Pricing — `aws.amazon.com/transcribe/pricing`	Precios oficiales
`.claude/hooks/telegram-commander.js` (lineas 2009-2097)	Codebase del proyecto
`.claude/hooks/telegram-image-utils.js`	Codebase del proyecto
`.claude/hooks/telegram-config.json`	Configuracion del proyecto